Time Complexity

Question 1: Why are constants and lower orders ignored in asymptotic analysis?

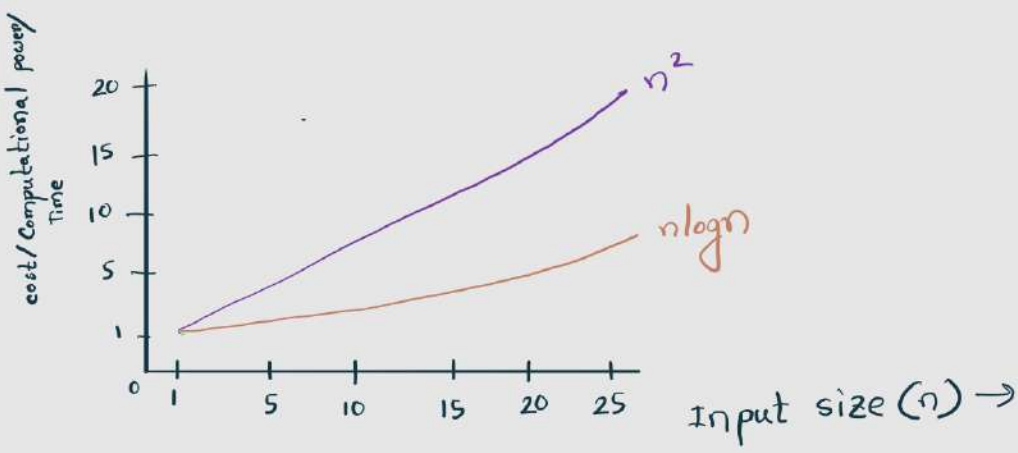
Ans: Asymptotic analysis deals with growth with respect to n.
It does not refer to the exact runtime.
So, if growth function is : $n^2 + 10n + 50 \longrightarrow O(n^2)$

$$\boxed{1 < \log n < n < n\log n < n^2 < n^3 < n^4 < 2^n < 3^n < n!}$$

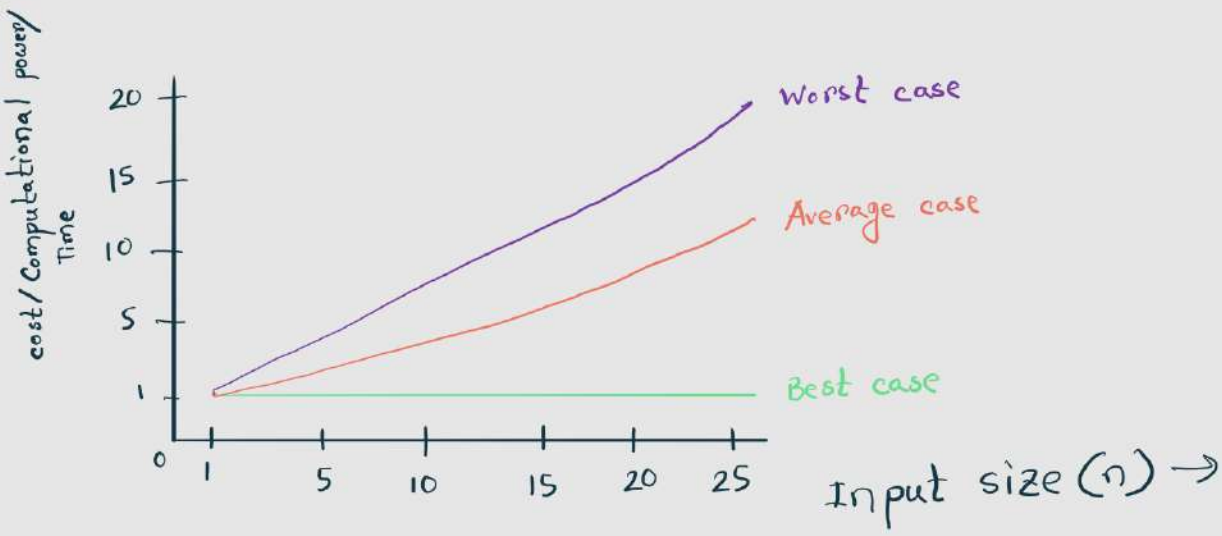
Less Time (Better) — More Time (Worse)

Question 2: Which is faster n^2 or $n\log n$?

Ans: $n\log n$ is faster as its growth is less when the value of n is increased



(3)



Summer 22:

1. a. CO7 **Calculate** the time complexity of the following function:

```
worstCase(n):
    int i, j, k, a, b, sum
    for ( i = 0; i < n; i = i + 3)
        for ( j = n; j >= 1; j = j / 5)
            for ( k = 1; k <= n; k = k * 5)
                sum = a + b
```

$O(n)$
$O(\log n)$
$O(\log n)$

$O(n(\log n)^2)$

Fall 23:

Explain the time complexity of the following code snippet in regards of the Big-O notation:

```
for (i=0; i<n; i+=4) {
    for (j=1; j<n; j*=2) {
        for (k=0; k<30; k++) {
            print("Am I still not 30?!!");
        }
        print("Why, God, why? We had a Deal!");
        for (m=n; m>0; m-=2) {
            print("Could you BE more dramatic?");
        }
    }
}
```

$O(n)$ ✓
$O(\log n)$ ✓
$O(1)$
$O(n)$ ✗

$O(n^2 \log n)$

Summer 23:

1 a. CO2 **Find** the time-complexity of the following task in terms of number of students. **03**

You are given a student attendance sheet. Each student has a unique integer ID. You have to count the number of students having an ID which is divisible by 3. The list is sorted but the IDs are not necessarily consecutive. So you check each ID one by one. → $O(n)$

b. CO2 **Write** the asymptotic time complexity of the following code snippet. Show your works/reasoning. **02**

```
for i in range (1,n)
    j= 1
    while j*j < i
        j= j+1
```

$O(n)$
$O(\sqrt{n})$
$O(n\sqrt{n})$

b. CO2 **Write** the asymptotic time complexity of the following code snippet. Show your works/reasoning. **02**

```
for i in range (1,n)
    j= 1
    while j < i*i
        j= j+1
```

$O(n)$
$O(n^2)$
$O(n^3)$

Important

```
for(i=1 ; i<n ; i++){
    if(i==5){
        break
    }
}
```

Ans:

$O(n)$ ✗

$O(1)$ ✓ → n এর value যতোই বাড়ুক, always i==5 হলে break করবে। n বাড়লেও no growth

Spring 25:

1 a. CO2 Analyze the following code snippet and **express** the time complexity using the Big O-notation. You must mention the time complexity for each loop. **04**

```
int b = 0;
for (int i = n; i > 0; i -= 3) {
    for (int j = 0; j <m; j += 3) {
        if (j == 50) {
            break;
        }
        b += i + j;
    }
}

for (int k = 1; k < n; k *= 3)
    for (int m = n; m > n; m /= 5) {
        b += k + m;
    }
}
return b;
```

$O(n)$
$O(m)$
$O(nm)$ ✗

$O(nm)$

$O(\log n)$
$O(\log)$
$O(\log n^2)$

Award 5 out of 100 student → $O(1)$

Sorting

① Bubble

5 1 4 2 8

1 5 4 2 8

1 4 5 2 8

1 4 2 5 8

1 4 2 5 8

Time complexity:

Best: $O(n)$

Avg/Worst: $O(n^2)$

Space Complexity:

$O(1)$

Inplace: Yes

Stable: Yes

Suitable: Small array

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n-1):
        for j in range(n-i-1):
            if arr[j] > arr[j+1]
                temp = arr[j]
                arr[j] = arr[j+1]
                arr[j+1] = temp
```

② Selection

64 25 12 22 11

64 [25 12 22 11]

11 25 [12 22 64]

11 12 25 [22 64]

11 12 22 25 [64]

Time complexity:

Best/Avg/Worst: $O(n^2)$

Space Complexity:

$O(1)$

Inplace: Yes

Stable: No

Suitable: Small array

```
def selection_sort(arr):
    n = len(arr)
    for i in range(1, n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
```

swap (arr[i], arr[min_idx])

4 [2 4 3]

2 4 [4 3]

2 3 4 4

Insertion Sort

j key

23] [1 10 5 2]

j key

1 23] [10 5 2]

j key

1 10 23 [5 2]

j key

1 5 10 23] [2

1 2 5 10 23

Time complexity:

Best: $O(n)$

Avg/Worst: $O(n^2)$

Space Complexity:

$O(1)$

Inplace: Yes

Stable: Yes

Suitable: Small array

key = arr[i]
= 5

```
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]    # store
        j = i-1
        while j>=0 and key< arr[j]:
            arr[j+1] = arr[j]
            j = j-1
        arr[j+1] = key
```

Binary Search

sorted

$O(\log n)$

l	mid	r
7	12	34	58	87

```
boolean binarySearch (A[], target)
        l = 0
        r = n-1
        while (l<=r)
            mid = (l+r)/2

            if (A[mid] == target) return true;

            else

                if (target < A[mid]) r = mid-1
                else   l = m+1
```

Ternary Search:

$\log_3 n = \log_2 n \times \log_3 2$

Ternary Search:

Find 31

3	5	7	12	15	23	27	31	45	56	70	98

Left Mid1 Mid2 Right

```
boolean TernarySearch (A[], l, r, item){
    while ( l<=r) {
        mid1 = l+(r-l)/3;
        mid2 = r-(r-l)/3;
        if (A[mid1]==item || A[mid2]==item) return true;
        else {
            if (item<A[mid1]) r = mid1-1;
            else if (item>A[mid2]) l = mid2+1;
            else  l = mid1+1;  r = mid2-1;
        }
```

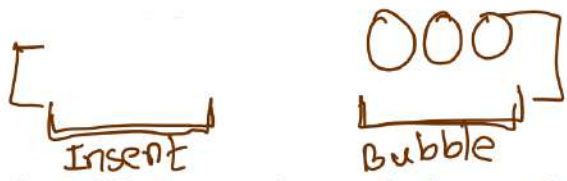
Spring 23

2 a. CO4 You are given an array containing N distinct integers in a wave-like sequence. Meaning, the numbers in the beginning are in ascending order, and after a specific position, they are in descending order. For example: [1, 3, 4, 5, 9, 6, 2, -1]
You have to find the maximum number of this sequence. Can you devise an efficient algorithm such that the time complexity will be less than $O(N)$?

a) **Present** your solution idea as a pseudocode/ python code/ flowchart/ step-by-step instructions/ logical explanation in one-two paragraphs. **04**

b) **Write** the time complexity of your algorithm. **01**

O(logn)



b. CO2 In case of sorting an array in ascending order, *Bubble Sort* extends a sorted subarray at the rightmost side and, *Insertion Sort* extends a sorted subarray at the leftmost side. Your friend, Jimmy was told to find the first five largest and smallest numbers from a list of N distinct integers ($N > 10$). To solve the task, he modified the Bubble sort and Insertion sort algorithm for only 5 iterations and used the rightmost and leftmost 5 numbers as the 5 largest and smallest numbers respectively. Do you support his strategy? **Explain** with logical reasons. **02**

[1, 3, 4, 5, 9, 6, 2, -1]

```
int binarySearch(A[])
    l = 0
    r = n-1
    while (l<=r)
        mid = (l+r)/2
        if (A[mid]>A[mid-1] && A[mid]>A[mid+1]) return A[mid]
        else
            if (target < A[mid]) r = mid-1
            else l = m+1
```

Fall 24:

b. Given an array of **N** elements, you are required to find the **peak element** in it. An element is considered to be a peak if its value is greater than the value(s) of its adjacent element(s) i.e. immediate left and immediate right. If there is only one adjacent element, we take only that in consideration. And the array will contain **exactly one** peak element.

Now answer the following questions.

CO1 i. **Determine** whether each of the following arrays matches the above stem-mentioned conditions: **02**

1. [1, **5**, 4, 3, 2] True
2. [1, **3**, 2, **5**, 4] False
3. [**5**, 2, 1, 3, **4**] False
4. [1, 2, 3, 4, **5**] True

[N.B.: For each array, it is enough to write only True/False in the answer script.]

CO3 ii. **Present** an algorithm with a time complexity lower than **O(N)** to solve the stem-mentioned problem using pseudocode/ programmable code/ step-by-step logical instructions. **03**

CO2 iii. If the array contains **more than one** peak element, will the presented algorithm in question (ii) still be able to find any peak element? **Explain** briefly. **01**

r

[1 5 4 3 2]

0 1

```
int binarySearch(A[])
    l = 0
    r = n-1
    while (l<=r)
        mid = (l+r)/2

        if (A[mid] < arr[mid+1]) left = mid+1
        else
            right = mid

    return arr[left]
```

Total Marks : 10

Jack loves to play with integers. He created a list of n integers where the even indices hold numbers in decreasing order and the odd indices hold numbers in increasing order. For example, this is a list of n=8 integers Jack made.

Index	0	1	2	3	4	5	6	7
Number	23	2	19	3	7	11	5	13

[Explanation:
The indices 1, 3, 5, and 7 have numbers 2, 3, 11, and 13 in increasing order.
The indices 0, 2, 4, and 6 have numbers 23, 19, 7 and 5 in decreasing order.]

You sorted the the list in quadratic time [O(n^2)]. To your utter surprise, Jack replies, "It could be sorted in linear time".

1) ***[CO2]* Show** the steps how you sorted the list using a suitable algorithm. Mention the name of the algorithm. — 3
2) ***[CO2]* Describe** how Jack sorted the list in linear time. Show the steps too.— 4 O(n)
3) ***[CO3]*** Jack wants to add the number 15 to your sorted list from (1). He wants you to come up with an efficient idea about how he is going to find the accurate index position for 15 in the list.
   Describe how you are going to assist him in this. **Show** the steps too. — 3

Bubble
Select
Insertion

(2) Step 1: Seperate even and odd
Step 2: Reverse even
Step 3: Merge

(3) Binary

2 Consider a farm with **n** sheep of two colors, say, white sheep and black sheep. The sheep are arranged in their barn in such a way that all white sheep are kept first, followed by all black sheep. Each of the sheep is assigned a serial number. **You are to find the serial number of the first black sheep.**

a. CO1 **Describe** how to convert the given information into an array of integers. **01**

b. CO1 Given the array of integers from Q(2a), **present** a method with a time complexity **less than O(n)** to solve the mentioned problem in the paragraph. Use pseudocode/ executable code/ step-by-step logical instructions to show your method. **01**

(a) White → 0

Black → 1

For example : 0 0 0 0 0 1 1

low mid high

mid ans

(b)
```
while (low <= high)
    mid = (low + high)/2

    if (arr[mid] == 1)
        ans = mid
        high = mid-1

    else  low = mid+1

return ans
```

2 a. Your friend gave you an integer (`key`) and asked you to find its integer square root without using built-in functions like `sqrt()` or exponentiation. For example, if the input is 25, the output should be 5 because $5^2 = 25$. If the square root is not an integer, return the floor value of the actual square root. For example, for an input of 10, the output should be 3.

Your friend's initial approach is to use linear search (time complexity $O(n)$) as shown below.

```
def LinearSearchToFindSquareRoot(key):
    result = -1
    for i in range(1, key, 1):
        if i * i <= key:
            result = i
    return result
```

Now to find the square root he is working on a possible solution **P,** which will result in time complexity $O(\sqrt{n})$. As an algorithm student, you propose another algorithm **R** within time complexity $O(log_2 N)$, which is much faster than $O(\sqrt{n})$.

CO2	i.	Present the **solution your friend** was trying to achieve **(Algorithm P)** with pseudocode/programmable code/step-by-step logical instructions.	03
CO3	ii.	Present your proposed solution **(Algorithm R)** with pseudocode/programmable code/step-by-step logical instructions.	04

(i)

(ii)
```
left = 1
right = key
ans = 0
while (left <= right)
        mid = (left + right) // 2

        if (mid * mid <= key)
                ans = mid
                left = mid + 1

        else right = mid - 1

return ans
```

2 a. CO3 You are given an array containing N distinct integers in ascending order. You have to find out the first missing number from the increasing sequence.

For example: [3, 4, 5, 6, 9, 10]. The first missing number in this sequence is 7.

[Explanation : If A_i and A_{i+1} are two consecutive integers from the list and their difference $(A_{i+1} - A_i)$ is more than 1, then we say A_i+1 is a missing number.]

Now that you have understood the problem completely, you are trying to solve it efficiently. Your friend, Jack, said that by iterating through all the numbers from the beginning, you can find the first missing number in linear time $[O(N)]$. But, you need a more efficient one. Now, **answer** the questions below.

a) If $N = 9$ and the maximum number (obviously the last-indexed one) in the array is 15, which one among 6, 8, 10 might be the first number (obviously the minimum one) in the array? **01**

b) Explain why you discarded the other two numbers in question no. (a). **01**

c) Assume $N = 7$ and the minimum and maximum numbers are 2 and 12 respectively. You found the middle-index value to be 5. On which side (left/right) of 5, do you expect to find the first missing number? Only mention 'left' or 'right'. **01**

d) Explain why you discarded the other half of the array in question no. (c). **01**

e) Assume $N = 7$ and the minimum and maximum numbers are 8 and 19 respectively. The missing elements are 11, 13, 14, 17, 18. What will be the value of the middle-index element? Determine the absolute difference between the middle-index value and the maximum number. **02**

N = 9

(a)

6 7 8 9 10 11 13 14 15

First Last

N = (Last − First) + 1

9 = (15 − First) + 1

First = 7

Ans: 6

(b)

2 3 4 5 6 7 8 9 10 11 12

Left Right

Right

2 a. CO1 You are given an array A consisting of N integers. Some of the numbers are positive, and some are negative. For example $A: \{15, -13, 3, -4, 5, -6\}$. **05**

You have to organize the array in a way that all the negative numbers come to the front and all positive numbers are pushed to the back. For example, the array A from above becomes $Org_A: \{-6, -4, -13, 15, 3, 5\}$. *Note that numbers don't need to be sorted here.*

Propose a linear-time $[O(N)]$ algorithm with constant $[O(1)]$ space complexity. **Present** your algorithm as pseudocode or executable code, or step-by-step logical instructions.

If unable, show a simulation of any algorithm (no limit on the time or space complexity) on the given example array A for 40% marks.

b. CO1 After solving Q2(a), you want to find the index of the leftmost positive number in the organized array Org_A. Present a modification of Binary Search to achieve this. **Write** either pseudocode or executable code. **03**

(a)

[15, -13, 3, -4, 5, -6]

L R

```
left = 0
right = n-1
while (left < right)
    if A[left] < 0 :
        left++
    else if A[right] >= 0
        right --
    else :
        swap(A[left++], A[right--])
```

(b) [-6, -4, -13, 15, 3, 5]

Modified binary Search

(b)

```
while (low <= high)
    mid = (low + high)/2
    if (arr[mid] == 1)
        ans = mid
        high = mid-1
    else low = mid+1
return ans
```